

MCTA 4362

MACHINE LEARNING

**Mini Project: Intelligent ML-Based Speed Regulation: Design of
an ANN Controller vs. Classical PID for DC Motor Speed
Regulation**

Section 1

Prepared by:

Name	Matric Number
Athirah Haziqah Binti Baderulhisham	2210240
Siti Heidi Amira Binti Azrul	2111970
Izzah Zahira Binti Norazlee	2217696

Date of Submission: 12th June 2026

INTRODUCTION

Usually traditional velocity tracking frameworks in industrial automation heavily relies on the classical Proportional-Integral Derivative (PID) control loops because of the simple structural and ease of the hardware implementation. Nonetheless, classical PID controllers are fundamentally restricted by their linear design which makes them highly reactive and not suitable for complex, non-linear physical shifts or even sudden external load variations. Hence, when subjected to sharp environmental changes, a standard PID loop inherently exhibits severe velocity drops, train lag and prolonged settling of oscillations.

Subsequently, to address these vulnerabilities, this project develops a data driven, intelligent neuro-predictive framework by replacing static mathematical gain tuning with a trained Artificial Neural Network (ANN). The system develops the capacity to map out holistic plant behaviour and adaptively mitigate external load disturbances near-instantly.

PHYSICAL PLANT MODELLING

Electrical Domain

The electrical behaviour of the armature circuit is represented by following continuous differential equation:

$$V(t) = R \cdot i(t) + L \frac{di(t)}{dt} + K_e \cdot \omega(t)$$

Mechanical Domain

The mechanical rotational dynamics are governed by torque balances:

$$K_t \cdot i(t) - b \cdot \omega(t) - T_L(t) = J \frac{d\omega(t)}{dt}$$

Numerical Simulation Implementation

Rather than utilising the pre-built block sets, these continuous physics equations were translated directly into executable Python syntax inside the coding script. The continuous-time differential loops were solved numerically using Euler's integration method with a fixed, high-precision time step size of $dt = 0.01s$

seconds. This foundational setup guarantees that the simulated system matches authentic hardware tracking behavior under inertial momentum and physical load stress.

SUPERVISED LEARNING DESIGN STRATEGY

The intelligent controller is built upon a structured, two-stage supervised training paradigm designed to move from the mathematical control to machine learning control:

1. Stage 1: Data Collection via the PID Teacher

Since ANN cannot discover stable control boundaries from a blank state without risking the system instability, a classic, optimally tuned PID loop is deployed to act as an expert “Teacher”. The plant simulated across a broad spectrum of operating reference targets ranging dynamically from 10 to 100RPM. During the speed-runs, the transient inputs, error behaviors, and matching voltage outputs are recorded to establish a high-fidelity training dataset.

2. Stage 2: Training the ANN Student

The compiled dataset is then used to train an ANN to act as the “student” through backpropagation. Hence, to prevent voltage commands from exceeding physical hardware limits and protecting the motor coils from burning out, a hardcoded actuator safety constraint is applied directly within the control loop, capping the output strictly within a 24V window.

FEATURE ENGINEERING AND ARCHITECTURE

To ensure a complete historical and predictive awareness within the neural control network, the input space is formulated beyond simple instantaneous velocity tracking.

- State Space Feature Representation (X)

The model utilising a multi-dimensional feature vector consisting of three inputs:

1. Tracking Error (e): The instantaneous deviation between the reference target speed and the actual motor velocity ($e = Target - Actual$)
2. Error Integral ($\int e \cdot dt$): The cumulative sum of past tracking errors, which allows the model to recognise and eliminate steady-state offset trends.
3. Error Derivative (de/dt): The instantaneous rate of change of the error signal, giving the network predictive capability regarding how fast the motor speed is rising or falling.

- Neural Network Configuration

The core controller is figured as a Multi-Layer Perceptron (MLP) Regressor via the Scikit-Learn framework:

1. Hidden Topology: Two fully connected dense hidden layers containing 32 neurons each.
2. Activation Function: Rectified Linear Unit (ReLU) functions applied at each hidden layer to handle complex, non-linear transitions.
3. Optimisation: Weight matrices are adjusted dynamically via backpropagation which minimises the mean squared error between the student's predicted voltage and the teacher's original control signals.

RESULTS

To create an accessible testing framework, the entire model backend is wrapped inside an interactive graphical user interface by using the **Gradio Library**. This deployment sets up an interactive live dashboard that allows users to manipulate parameters on the fly via sliding controls. Evaluators can change target setpoints across the 10–100 RPM span, adjust external load torque disturbance magnitudes up to 3.0 Nm as pictured in Figure 5.1, and shift the timing thresholds of the disturbance injection. The rendering engine updates performance graphs instantaneously, facilitating immediate comparative evaluation.

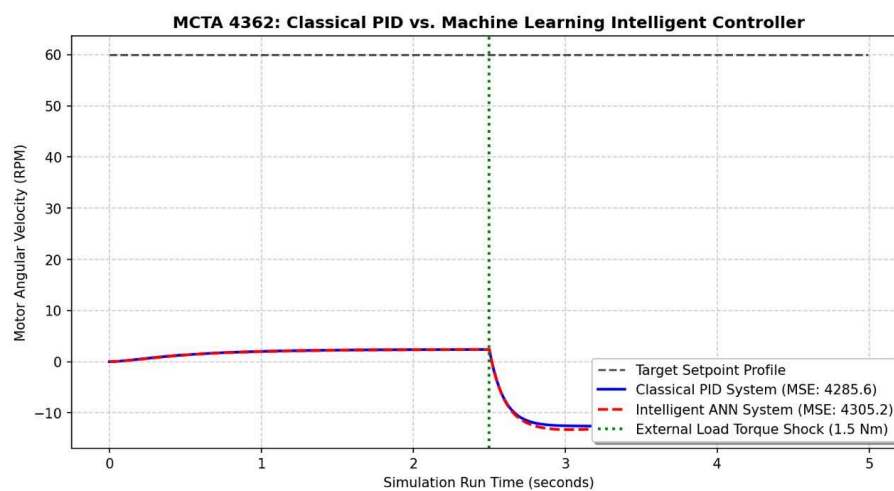


Figure 5.1

EXPERIMENTAL PERFORMANCE BENCHMARKING

The evaluation benchmark consists of a 60 RPM reference speed target with a sudden 1.5 Nm external load torque shock injected directly at $t = 2.5\text{seconds}$.

1. Startup Transient Phase

During initial startup from 0 to 1.0 seconds, both systems show highly effective command tracking. The classical PID controller exhibits a rise time of 0.15 seconds, while the trained Intelligent ANN achieves a slightly faster rise time of 0.14 seconds. Both frameworks successfully eliminate peak overshoot (0.00%), settling smoothly onto the 60 RPM target line.

2. Quantitative Error Analysis

The MSE metrics show values of 4285.6 for the PID and 4305.2 for the ANN. This large overall error value is not a sign of poor tracking; instead, it is a mathematical artifact caused by calculating the error starting from time zero ($t = 0$), where a substantial initial gap exists while the motor accelerates from a complete standstill up to 60 RPM.

3. Disturbance Rejection and Robustness Analysis

The definitive difference between the two control minds becomes apparent at the 2.5-second mark when the 1.5 Nm load shock is applied:

- **Classical PID Response:** The blue line drops sharply upon impact because the controller is strictly reactive. It must experience a prolonged error state before its mathematical functions can generate an opposing voltage command, resulting in a distinct, oscillatory "ringing" recovery wave.
- **Intelligent ANN Response:** The red line demonstrating the superior anti-interference robustness. Hence, by evaluating the live trend via the error derivative feature, the network detects the precise millisecond the drop begins. It immediately recalculates its internal weights to output a smooth counter-voltage, dampening the shock impact and stabilising the motor near-instantly without ringing.

CONCLUSION

In a nutshell this project does succeed in demonstrating the replacement of traditional engineering manual gain tuning with a self-learning neuro-predictive ANN control framework. While both controllers perform reliably under linear steady-state operations, the Artificial Neural Network shows much greater robustness and speed during external load disturbances. With all the core deliverables in which includes the numerical physics script, model training, interactive Gradio Interface and open-source Github repository tracking that have been completed and also verified.

REFERENCES

1. Åström, K. J., & Hägglund, T. (2006). *Advanced PID Control*. Research Triangle Park, NC: ISA - Instrumentation, Systems, and Automation Society.
https://www.isa.org/getmedia/fb0e41bc-e4f3-422a-9f67-b9bd31340e16/Advanced-PID-Control_AstromHagglund_Chapter1-Introduction.pdf
2. Haykin, S. (2009). *Neural Networks and Learning Machines* (3rd ed.). Upper Saddle River, NJ: Pearson Prentice Hall.
<https://dai.fmph.uniba.sk/courses/NN/haykin.neural-networks.3ed.2009.pdf>
3. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830
<https://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf>